CLASE 10

Lenguajes de Programación Robótica y ROS 2

Guía de Autoaprendizaje — versión 3

Robótica Industrial · FPTEN27 · Universidad Metropolitana

Esta guía está diseñada para que puedas estudiar y reproducir los temas de la Clase 10 de forma independiente. Incluye un mismo programa escrito en tres lenguajes (RAPID, KRL y Python), la representación de la orientación con ángulos de Euler y cuaterniones, la arquitectura de ROS 2, la demostración con turtlesim, scripts de figuras con control en lazo cerrado, dos robots controlados por un mismo nodo y un servicio programado por ti. Los números entre corchetes, como [1], remiten a las referencias del final.

1. Lenguajes de Programación Robótica

1.1 ¿Por qué existen múltiples lenguajes?

Cada fabricante de robots industriales desarrolló su propio lenguaje propietario. El resultado: un programa escrito para ABB no puede ejecutarse en KUKA sin reescribirse. Python y ROS 2 surgen como alternativas abiertas y portables.

Fabricante	Lenguaje	Extensión	Característica
ABB	RAPID	.mod	Módulos y cuaterniones de orientación
KUKA	KRL	.src	Variables $ del sistema y ángulos de Euler
Fanuc	TP / Karel	.ls / .pc	Teach pendant y alto nivel
Universal Robots	URScript	.script	Similar a Python pero propietario
Cualquier marca	Python + ROS 2	.py	Abierto, portable, estándar de facto

1.2 Un mismo programa en tres lenguajes

Los tres códigos de esta sección hacen exactamente lo mismo, con los mismos cuatro puntos y la herramienta apuntando hacia abajo:

1. **PTP** (movimiento articular) hasta P1.
2. **Lineal**: línea recta de P1 a P2.
3. **Circular**: arco que empieza en P2, pasa por P3 y termina en P4.

Punto	X (mm)	Y (mm)	Z (mm)	Papel en el programa
P1	150	-50	50	Destino del PTP
P2	200	-50	50	Destino de la lineal e inicio del arco
P3	250	0	50	Punto de paso del arco
P4	200	50	50	Final del arco

El arco es un semicírculo de 50 mm de radio con centro en (200, 0). Toda instrucción circular necesita **tres puntos**: el de inicio (donde ya está el robot), uno de paso y el final [8].

1.3 RAPID — ABB

```
MODULE Ejemplo_ABB
    ! ============================================================
    !  RAPID (ABB) - PTP, Lineal y Circular
    !  Comentarios con signo de exclamacion
    ! ============================================================

    ! Puntos: [[X,Y,Z] mm], [cuaternion q1..q4], [configuracion], [ejes externos]
    CONST robtarget P1 := [[150,-50,50],[0,1,0,0],[0,0,0,0],[9E9,9E9,9E9,9E9,9E9,9E9]];
    CONST robtarget P2 := [[200,-50,50],[0,1,0,0],[0,0,0,0],[9E9,9E9,9E9,9E9,9E9,9E9]];
    CONST robtarget P3 := [[250,  0,50],[0,1,0,0],[0,0,0,0],[9E9,9E9,9E9,9E9,9E9,9E9]];
    CONST robtarget P4 := [[200, 50,50],[0,1,0,0],[0,0,0,0],[9E9,9E9,9E9,9E9,9E9,9E9]];

    PROC Main()
        ! 1) PTP: movimiento articular hasta P1
        MoveJ P1, v100, z10, tool0;

        ! 2) Lineal: linea recta de P1 a P2
        MoveL P2, v100, z10, tool0;

        ! 3) Circular: arco desde P2, pasando por P3, hasta P4
        MoveC P3, P4, v100, fine, tool0;
    ENDPROC

ENDMODULE
```

- **`MODULE`** … **`ENDMODULE`**: todo el código vive dentro de un módulo, que agrupa los datos y las rutinas.
- **`!`** inicia un comentario. Cada instrucción termina en **`;`** y la asignación usa **`:=`**.
- **`CONST robtarget`** declara un punto constante. Un robtarget tiene cuatro bloques: posición [x, y, z] en mm, orientación en cuaternión [q1, q2, q3, q4], configuración del brazo [cf1, cf4, cf6, cfx] y ejes externos [5].
- **`9E9`** indica que ese eje externo no se usa [4]. En este ejemplo, la configuración `[0,0,0,0]` es un valor de relleno.
- **`PROC Main()`** … **`ENDPROC`** define el procedimiento que se ejecuta al arrancar.
- Cada movimiento lleva sus parámetros en la misma línea: destino, velocidad (`v100`, 100 mm/s para el TCP), zona (`z10`: pasa a 10 mm del punto sin detenerse; `fine`: se detiene exactamente en el punto) y herramienta (`tool0`, la herramienta predefinida) [4].
- **`MoveC P3, P4`**: primero el punto de paso (CirPoint) y luego el final (ToPoint) [4].

1.4 KRL — KUKA

```
DEF Ejemplo_KUKA()
; ============================================================
;  KRL (KUKA) - PTP, Lineal y Circular
;  Comentarios con punto y coma
; ============================================================

DECL POS P1, P2, P3, P4      ; declaracion explicita de tipos

BAS(#INITMOV,0)              ; inicializacion obligatoria
$VEL.CP = 0.1                ; velocidad de trayectoria (m/s)

; Puntos: X,Y,Z en mm y orientacion A,B,C (angulos de Euler, grados)
P1 = {X 150, Y -50, Z 50, A 0, B 0, C 180}
P2 = {X 200, Y -50, Z 50, A 0, B 0, C 180}
P3 = {X 250, Y   0, Z 50, A 0, B 0, C 180}
P4 = {X 200, Y  50, Z 50, A 0, B 0, C 180}

; 1) PTP: movimiento articular hasta P1
PTP P1

; 2) Lineal: linea recta de P1 a P2
LIN P2

; 3) Circular: arco desde P2, pasando por P3 (auxiliar), hasta P4
CIRC P3, P4

END
```

- **`DEF` … `END`**: estructura más plana que RAPID, sin módulos.
- **`;`** inicia un comentario. Ojo: en RAPID el `;` cierra cada instrucción, en KRL abre un comentario. Las instrucciones de KRL no llevan terminador y la asignación usa `=`.
- **`DECL POS`** declara variables de tipo posición. KRL exige declarar los tipos antes de cualquier instrucción ejecutable.
- **`BAS(#INITMOV,0)`** carga los parámetros de movimiento por defecto; va al inicio del programa [6].
- **`$`** marca las variables del sistema. **`$VEL.CP`** es la velocidad de la trayectoria cartesiana, en **metros por segundo** (0.1 m/s = 100 mm/s) [6]. Se fija una vez y aplica a los movimientos siguientes.
- **`{X 150, Y -50, Z 50, A 0, B 0, C 180}`**: posición en mm y orientación con ángulos de Euler A, B, C en grados (ver sección 2) [7].
- **`PTP`**, **`LIN`** y **`CIRC P3, P4`** (punto auxiliar, punto final) [5].

1.5 Python — API de RoboDK

```
# ============================================================
#  Python (API de RoboDK) - PTP, Lineal y Circular
#  Comentarios con numeral
# ============================================================
from robodk.robolink import Robolink, ITEM_TYPE_ROBOT
from robodk.robomath import transl, rotx, pi

RDK   = Robolink()                          # conectar con RoboDK
robot = RDK.Item('', ITEM_TYPE_ROBOT)       # el robot de la estacion

# Puntos: pose 4x4 = traslacion (mm) x rotacion (herramienta hacia abajo)
P1 = transl(150, -50, 50) * rotx(pi)
P2 = transl(200, -50, 50) * rotx(pi)
P3 = transl(250,   0, 50) * rotx(pi)
P4 = transl(200,  50, 50) * rotx(pi)

robot.setSpeed(100)                         # velocidad lineal (mm/s)

# 1) PTP: movimiento articular hasta P1
robot.MoveJ(P1)

# 2) Lineal: linea recta de P1 a P2
robot.MoveL(P2)

# 3) Circular: arco desde P2, pasando por P3, hasta P4
robot.MoveC(P3, P4)
```

- Python no trae comandos de robot propios: se **importan** de una librería. `robolink` conecta con RoboDK y `robomath` aporta la matemática de poses. `#` inicia un comentario.
- **`RDK.Item('', ITEM_TYPE_ROBOT)`** toma el robot de la estación [8].
- Cada punto es una **matriz homogénea 4×4**, como las de la Clase 8: `transl(x, y, z)` es la traslación y `rotx(pi)` una rotación de 180° alrededor de X, con el ángulo en radianes [8]. El `*` es el producto de matrices.
- **`robot.setSpeed(100)`** fija la velocidad lineal en mm/s, una sola vez [8].
- **`robot.MoveJ(P1)`**, **`robot.MoveL(P2)`** y **`robot.MoveC(P3, P4)`** son métodos del objeto robot: `objeto.método(argumentos)` [8].

1.6 Comparación

	RAPID (ABB)	KRL (KUKA)	Python (RoboDK)
Bloque del programa	`MODULE` + `PROC`	`DEF … END`	Ninguno obligatorio
Comentario	`!`	`;`	`#`
Fin de instrucción	`;`	Nada	Nada
Asignación	`:=`	`=`	`=`
Orientación	Cuaternión `[0,1,0,0]`	Euler `A 0, B 0, C 180`	Matriz `rotx(pi)`
Velocidad	En cada movimiento, mm/s	`$VEL.CP` una vez, m/s	`setSpeed()` una vez, mm/s
Circular	`MoveC paso, final, …`	`CIRC paso, final`	`MoveC(paso, final)`

> ★ **Observación clave**
> Las coordenadas de los cuatro puntos son idénticas en los tres programas, y las tres orientaciones son el mismo giro de 180° alrededor de X escrito de tres formas. El robot va exactamente al mismo lugar: solo cambia el idioma en que se le da la orden.

> ℹ **¿Cómo genera RoboDK los programas de cada marca?**
> Mediante un **post procesador**: un archivo que traduce el programa neutro de la estación al lenguaje de cada fabricante (por ejemplo, `ABB_RAPID_IRC5.py` o `KUKA_KRC5.py` en la carpeta Posts de RoboDK). El programa se genera seleccionándolo en el árbol de la estación y presionando F6 [8].
> La versión gratuita de RoboDK limita la generación a programas de 50 líneas [8].

2. Orientación del efector: Euler y cuaterniones

2.1 Ángulos de Euler (KUKA)

KUKA describe la orientación con tres ángulos en grados: **A** gira alrededor del eje z, **B** alrededor del nuevo eje y, y **C** alrededor del nuevo eje x (convención z-y'-x'') [7]. Son los ángulos de Euler ZYX de Lynch & Park [1, Ap. B.1].

2.2 Cuaterniones (ABB y ROS)

Cualquier orientación se puede lograr con **un solo giro** de ángulo θ alrededor de un eje fijo [2, sec. 3.1.4]. El cuaternión guarda ese eje y ese ángulo en cuatro números [1, Ap. B.3, ec. B.9]:

$$q = [\ \cos(\theta/2)\ ,\ \ kx\cdot\sin(\theta/2)\ ,\ \ ky\cdot\sin(\theta/2)\ ,\ \ kz\cdot\sin(\theta/2)\ ]$$

- **q1 = cos(θ/2)** dice cuánto se gira: si vale 1 no hay giro; si vale 0 el giro es de 180°.
- **q2, q3, q4** dicen alrededor de qué eje: son las componentes x, y, z del eje de giro (un vector unitario), escaladas por sin(θ/2).
- Siempre se cumple $q1^2 + q2^2 + q3^2 + q4^2 = 1$, así que solo tres números son independientes [1, Ap. B.3].

- `q` y `−q` representan la misma orientación [1, Ap. B.3]: [0, 1, 0, 0] y [0, −1, 0, 0] son lo mismo.

Ejemplos, verificados con la función `Pose_2_ABB` de RoboDK [8]:

Orientación	Cuaternión [q1, q2, q3, q4]	Lectura
Sin giro	[1, 0, 0, 0]	q1 = 1: ángulo 0°
90° alrededor de Z	[0.7071, 0, 0, 0.7071]	cos 45° = 0.7071, eje Z
180° alrededor de X	[0, 1, 0, 0]	Herramienta hacia abajo (el de la sección 1)
180° en X y luego 90° en Z	[0, 0.7071, 0.7071, 0]	Un solo giro de 180° alrededor de la diagonal entre X e Y

> **ℹ ¿Por qué [0, 1, 0, 0] es la herramienta hacia abajo?**
> Es un giro de θ = 180° alrededor de X: q1 = cos(90°) = 0; el eje X es (1, 0, 0) y sin(90°) = 1, así que q2 = 1, q3 = 0, q4 = 0. Al girar 180° alrededor de X, el eje Z de la herramienta pasa de apuntar hacia arriba a apuntar hacia abajo. En KUKA, ese mismo giro se escribe A = 0, B = 0, C = 180.

2.3 Dos tipos de singularidad que no hay que confundir

Singularidad de representación (gimbal lock)

Los ángulos de Euler ZYX sí pueden representar **todas** las orientaciones. Pero cuando el ángulo del medio vale ±90° (B = ±90° en KUKA), la representación deja de ser única: hay infinitas combinaciones de A y C que describen la misma orientación [1, Ap. B.1]. El robot llega a esa orientación sin problema; lo que falla es la descripción con tres números. Toda representación de tres parámetros tiene necesariamente una singularidad, y el cuaternión la evita usando cuatro [1, Ap. B.3]. Por eso los cuaterniones son la representación preferida para cálculo [2, sec. 3.1.4] y ROS los adoptó como estándar [3, sec. 6.4.3].

Singularidad cinemática

Es una postura del **mecanismo** en la que el efector pierde la capacidad de moverse en una o más direcciones; matemáticamente, el Jacobiano deja de tener rango máximo [1, sec. 5.3]. Ejemplos: dos ejes de rotación colineales, o tres ejes paralelos y coplanares (brazo estirado) [1, sec. 5.3]. Cerca de ella, mantener la velocidad cartesiana pedida exige velocidades articulares cada vez mayores [1, sec. 5.4]. **Escribir la orientación con cuaterniones no la elimina**: depende de los ángulos de las articulaciones, no de la notación. Los movimientos lineales y circulares no pueden cruzar una singularidad [8]; un PTP no impone un camino cartesiano y suele ser la salida.

	De representación (gimbal lock)	Cinemática
¿Dónde está?	En la notación (Euler A, B, C)	En el mecanismo (ángulos θ)
¿Qué se pierde?	La unicidad de los números	Capacidad de moverse en alguna dirección
¿La resuelve el cuaternión?	Sí	No
¿A quién afecta?	A quien use 3 parámetros	A cualquier robot con esa geometría

3. ROS 2 — Robot Operating System

3.1 ¿Qué es ROS 2?

ROS 2 es un middleware para robots: una capa de software entre el sistema operativo (Linux) y las aplicaciones del robot. Su función principal es resolver la comunicación entre los distintos componentes de un sistema robótico sin importar el fabricante ni el lenguaje.

ROS 1 — Centralizado

Depende de un nodo maestro (roscore). Si falla, todo el sistema cae. No adecuado para sistemas críticos.

ROS 2 — Descentralizado (DDS)

Peer-to-peer. Ningún nodo puede bloquear a los demás. Ideal para multi-robot y aplicaciones críticas.

3.2 Nodo, Tópico y Mensaje

Nodo

Proceso independiente con una tarea específica. Cada componente del robot (cámara, sensor, motor) es un nodo separado. Un solo programa puede contener uno o más nodos [9].

★ Analogía del nodo

Departamentos de una empresa: ventas, contabilidad, logística. Cada uno hace su trabajo de forma independiente y se comunica con los demás a través de canales definidos.

Tópico

Canal de comunicación con nombre. Un nodo publica datos (publisher) y otro los recibe (subscriber). El publisher no sabe cuántos subscribers hay: comunicación desacoplada y asíncrona, de uno a muchos [9].

★ Analogía del tópico

Una emisora de radio transmite en una frecuencia (tópico) sin saber cuántos oyentes hay. Los oyentes se sintonizan en la frecuencia que les interesa.

Mensaje

Estructura de datos estándar que viaja por un tópico. Ejemplos:

- `geometry_msgs/Twist`: velocidad lineal y angular (robots móviles).
- `turtlesim/Pose`: posición x, y y orientación theta.
- `sensor_msgs/Image`: imagen de cámara.

3.3 Servicio

Los servicios usan un modelo de **llamada y respuesta**: un cliente envía una petición, el servidor la procesa y devuelve una respuesta. A diferencia de los tópicos, que entregan un flujo continuo, un servicio solo entrega datos cuando un cliente lo llama, y no conviene usarlo para llamadas continuas [9].

ℹ Un matiz: ¿síncrono o asíncrono?

El **patrón** es petición-respuesta: el cliente espera la contestación. Pero en Python, la llamada síncrona `call()` bloquea el programa hasta recibir la respuesta y puede dejarlo congelado sin ningún aviso. ROS 2 recomienda `call_async()`, que devuelve de inmediato un *future*: un valor que indica cuándo llegó la respuesta [9].

Los scripts de esta guía usan `call_async()` y luego esperan con `spin_until_future_complete()`.

4. Demo con turtlesim — Paso a paso

Turtlesim es la herramienta de aprendizaje más usada en ROS 2 para entender nodos, tópicos y mensajes de forma visual, sin hardware real.

⚠ Antes de empezar

Necesitas ROS 2 Humble en Ubuntu 22.04 (sección 9). En **cada** terminal nueva ejecuta primero:
`source /opt/ros/humble/setup.bash`

4.1 Lanzar turtlesim y control por teclado

T1

Terminal 1 — Lanzar la tortuga

```
ros2 run turtlesim turtlesim_node
```

Aparece la ventana azul con la tortuga. Este nodo escucha velocidades en /turtle1/cmd_vel y publica su posición en /turtle1/pose.

T2

Terminal 2 — Control por teclado

```
ros2 run turtlesim turtle_teleop_key
```

Usa las flechas para mover la tortuga. Este segundo nodo publica velocidades en el tópico.

4.2 Inspeccionar la arquitectura

Con las dos terminales anteriores corriendo, abre una tercera:

Comando	¿Qué hace?
`ros2 node list`	Lista los nodos activos: /turtlesim y /teleop_turtle [9]
`ros2 node info /turtlesim`	Muestra con quién se comunica ese nodo: suscriptores, publicadores, servicios y acciones [9]
`ros2 topic list`	Lista los tópicos: /turtle1/cmd_vel, /turtle1/pose, /turtle1/color_sensor…
`ros2 topic echo /turtle1/cmd_vel`	Muestra las velocidades en tiempo real: presiona una flecha y verás los números
`ros2 topic echo /turtle1/pose`	Posición (x, y) y orientación (theta) en tiempo real
`ros2 topic hz /turtle1/pose`	Frecuencia de publicación: la tortuga informa su pose unas 60 veces por segundo [9]
`ros2 topic echo /turtle1/color_sensor`	Color (R, G, B) del píxel bajo la tortuga; cambia al pasar sobre sus propios trazos
`ros2 service list`	Lista los servicios: /reset, /spawn, /turtle1/set_pen, etc.
`ros2 service call /reset std_srvs/srv/Empty`	Borra los trazos y regresa la tortuga al centro

4.3 Ver el grafo con rqt_graph

```
rqt_graph
```

Dibuja los nodos como óvalos y los tópicos como flechas, en el sentido en que viajan los datos (del nodo que publica al que se suscribe) [9]. Si algo cambia, presiona el botón de actualizar. En la sección 8 se explica cada nodo que aparece.

4.4 Control de velocidad manual

Detén el teleop (Ctrl+C en T2) y publica velocidades directamente:

Círculo

```
ros2 topic pub /turtle1/cmd_vel geometry_msgs/msg/Twist \
  "{linear: {x: 2.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 1.0}}"
```

`linear.x` = velocidad hacia adelante; `angular.z` = velocidad de giro. Valores constantes = círculo.

Giro en el lugar

```
ros2 topic pub /turtle1/cmd_vel geometry_msgs/msg/Twist \
  "{linear: {x: 0.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 2.0}}"
```

`linear.x` = 0 y `angular.z` ≠ 0: rotación sobre su propio eje, sin desplazamiento. Para detener: Ctrl+C en la terminal del pub.

ℹ ¿Por qué la tortuga sigue moviéndose?

Si no se indica otra cosa, `ros2 topic pub` publica el mensaje **repetidamente a 1 Hz**. Con la opción `--once` envía un solo mensaje [9].

Turtlesim descarta la orden de velocidad si no recibe una nueva en 1 segundo, para comportarse como un robot real, al que hay que enviarle órdenes a ritmo constante [10]. Por eso, con `--once` la tortuga se mueve un instante y se detiene. Es una medida de seguridad: si el control remoto se cae, el robot se detiene solo.

5. Scripts de Figuras Geométricas

ℹ Control por pose — lazo cerrado

Estos scripts leen el tópico /turtle1/pose en cada iteración para conocer la posición y orientación **reales** de la tortuga antes de publicar el siguiente comando. Es un lazo cerrado: medir, comparar con el objetivo y actuar. Es el mismo principio de los controladores PID.

Controlar por tiempo (lazo abierto: suponer que en X segundos la tortuga recorre Y distancia) falla en WSL2, porque la latencia variable acumula errores y deforma las figuras.

5.1 Cuadrado

Avanzar 2.5 unidades, girar exactamente 90° (π/2 rad) y repetir 4 veces. `girar_a(0.0)` orienta la tortuga a 0° antes de empezar para que el cuadrado quede alineado. `girar()` usa velocidad proporcional al error: frena al acercarse al ángulo objetivo.

```
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist
from turtlesim.msg import Pose
import math, time

class Cuadrado(Node):
    def __init__(self):
        super().__init__('cuadrado')
        self.pub  = self.create_publisher(Twist, '/turtle1/cmd_vel', 10)
        self.sub  = self.create_subscription(Pose, '/turtle1/pose', self.cb_pose, 10)
        self.pose = None
        time.sleep(1)

    def cb_pose(self, msg): self.pose = msg

    def esperar_pose(self):
        while self.pose is None: rclpy.spin_once(self, timeout_sec=0.05)

    def avanzar(self, distancia, velocidad=1.5):
        self.esperar_pose()
        x0, y0 = self.pose.x, self.pose.y
        msg = Twist()
        while True:
            rclpy.spin_once(self, timeout_sec=0.01)
            recorrido = math.sqrt((self.pose.x-x0)**2 + (self.pose.y-y0)**2)
            restante  = distancia - recorrido
            if restante <= 0: break
            msg.linear.x = min(velocidad, max(0.3, restante*2.0))
            self.pub.publish(msg)
```

```
        self.pub.publish(Twist())
        time.sleep(0.1)

    def girar(self, angulo_rad):
        self.esperar_pose()
        objetivo = self.pose.theta + angulo_rad
        msg = Twist()
        while True:
            rclpy.spin_once(self, timeout_sec=0.01)
            diff = objetivo - self.pose.theta
            while diff >  math.pi: diff -= 2*math.pi
            while diff < -math.pi: diff += 2*math.pi
            if abs(diff) < 0.002: break   # tolerancia ultra fina
            vel = max(0.2, min(2.0, abs(diff)*2.5))  # velocidad proporcional al error
            msg.angular.z = vel if diff > 0 else -vel
            self.pub.publish(msg)
        self.pub.publish(Twist())
        time.sleep(0.1)

    def girar_a(self, angulo_objetivo):
        self.esperar_pose()
        msg = Twist()
        while True:
            rclpy.spin_once(self, timeout_sec=0.01)
            diff = angulo_objetivo - self.pose.theta
            while diff >  math.pi: diff -= 2*math.pi
            while diff < -math.pi: diff += 2*math.pi
            if abs(diff) < 0.002: break
            vel = max(0.2, min(1.5, abs(diff)*2.5))
            msg.angular.z = vel if diff > 0 else -vel
            self.pub.publish(msg)
        self.pub.publish(Twist())
        time.sleep(0.1)

    def dibujar(self):
        self.girar_a(0.0)       # orientar a 0° antes de empezar
        for _ in range(4):
            self.avanzar(2.5)
            self.girar(math.pi/2)  # 90° exactos

def main():
    rclpy.init()
    nodo = Cuadrado()
    nodo.dibujar()
    nodo.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__': main()
```

```
ros2 service call /reset std_srvs/srv/Empty
nano ~/cuadrado.py   # pegar el código, guardar con Ctrl+O y salir con Ctrl+X
python3 ~/cuadrado.py
```

5.2 Triángulo equilátero

Avanzar, girar 120° (2π/3 rad) y repetir 3 veces. La misma lógica sirve para cualquier polígono regular cambiando el ángulo y las repeticiones.

```
import rclpy
from rclpy.node import Node
```

```
from geometry_msgs.msg import Twist
from turtlesim.msg import Pose
import math, time

class Triangulo(Node):
    def __init__(self):
        super().__init__('triangulo')
        self.pub  = self.create_publisher(Twist, '/turtle1/cmd_vel', 10)
        self.sub  = self.create_subscription(Pose, '/turtle1/pose', self.cb_pose, 10)
        self.pose = None
        time.sleep(1)

    def cb_pose(self, msg): self.pose = msg

    def esperar_pose(self):
        while self.pose is None: rclpy.spin_once(self, timeout_sec=0.05)

    def avanzar(self, distancia, velocidad=1.5):
        self.esperar_pose()
        x0, y0 = self.pose.x, self.pose.y
        msg = Twist()
        while True:
            rclpy.spin_once(self, timeout_sec=0.01)
            recorrido = math.sqrt((self.pose.x-x0)**2 + (self.pose.y-y0)**2)
            restante  = distancia - recorrido
            if restante <= 0: break
            msg.linear.x = min(velocidad, max(0.3, restante*2.0))
            self.pub.publish(msg)
        self.pub.publish(Twist())
        time.sleep(0.1)

    def girar(self, angulo_rad):
        self.esperar_pose()
        objetivo = self.pose.theta + angulo_rad
        msg = Twist()
        while True:
            rclpy.spin_once(self, timeout_sec=0.01)
            diff = objetivo - self.pose.theta
            while diff >  math.pi: diff -= 2*math.pi
            while diff < -math.pi: diff += 2*math.pi
            if abs(diff) < 0.002: break
            vel = max(0.2, min(2.0, abs(diff)*2.5))
            msg.angular.z = vel if diff > 0 else -vel
            self.pub.publish(msg)
        self.pub.publish(Twist())
        time.sleep(0.1)

    def dibujar(self):
        for _ in range(3):
            self.avanzar(3.0)
            self.girar(2*math.pi/3)  # 120° exactos

def main():
    rclpy.init()
    nodo = Triangulo()
    nodo.dibujar()
    nodo.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__': main()
```

```
ros2 service call /reset std_srvs/srv/Empty
nano ~/triangulo.py   # pegar el código, guardar con Ctrl+O y salir con Ctrl+X
python3 ~/triangulo.py
```

5.3 Estrella de 5 puntas

Avanzar, girar 144° a la **derecha** (velocidad angular negativa) y repetir 5 veces. 5 × 144° = 720° = dos vueltas completas: por eso se forma una estrella y no un pentágono.

```
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist
from turtlesim.msg import Pose
import math, time

class Estrella(Node):
    def __init__(self):
        super().__init__('estrella')
        self.pub  = self.create_publisher(Twist, '/turtle1/cmd_vel', 10)
        self.sub  = self.create_subscription(Pose, '/turtle1/pose', self.cb_pose, 10)
        self.pose = None
        time.sleep(1)

    def cb_pose(self, msg): self.pose = msg

    def esperar_pose(self):
        while self.pose is None: rclpy.spin_once(self, timeout_sec=0.05)

    def avanzar(self, distancia, velocidad=1.5):
        self.esperar_pose()
        x0, y0 = self.pose.x, self.pose.y
        msg = Twist()
        while True:
            rclpy.spin_once(self, timeout_sec=0.01)
            recorrido = math.sqrt((self.pose.x-x0)**2 + (self.pose.y-y0)**2)
            restante  = distancia - recorrido
            if restante <= 0: break
            msg.linear.x = min(velocidad, max(0.3, restante*2.0))
            self.pub.publish(msg)
        self.pub.publish(Twist())
        time.sleep(0.1)

    def girar(self, angulo_rad):
        self.esperar_pose()
        objetivo = self.pose.theta + angulo_rad
        msg = Twist()
        while True:
            rclpy.spin_once(self, timeout_sec=0.01)
            diff = objetivo - self.pose.theta
            while diff >  math.pi: diff -= 2*math.pi
            while diff < -math.pi: diff += 2*math.pi
            if abs(diff) < 0.002: break
            vel = max(0.2, min(2.0, abs(diff)*2.5))
            msg.angular.z = vel if diff > 0 else -vel
            self.pub.publish(msg)
        self.pub.publish(Twist())
        time.sleep(0.1)

    def dibujar(self):
        # Estrella de 5 puntas: girar 144° a la derecha en cada punta
        for _ in range(5):
```

```
            self.avanzar(3.5)
            self.girar(-(144*math.pi/180))

def main():
    rclpy.init()
    nodo = Estrella()
    nodo.dibujar()
    nodo.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__': main()
```

```
ros2 service call /reset std_srvs/srv/Empty
nano ~/estrella.py   # pegar el código, guardar con Ctrl+O y salir con Ctrl+X
python3 ~/estrella.py
```

5.4 Espiral creciente

La velocidad lineal aumenta en cada iteración mientras el giro es constante. La espiral se detiene cuando la tortuga se aleja más de 4.0 unidades del punto de inicio.

```
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist
from turtlesim.msg import Pose
import math, time

class Espiral(Node):
    def __init__(self):
        super().__init__('espiral')
        self.pub  = self.create_publisher(Twist, '/turtle1/cmd_vel', 10)
        self.sub  = self.create_subscription(Pose, '/turtle1/pose', self.cb_pose, 10)
        self.pose = None
        time.sleep(1)

    def cb_pose(self, msg): self.pose = msg

    def esperar_pose(self):
        while self.pose is None: rclpy.spin_once(self, timeout_sec=0.05)

    def dibujar(self):
        self.esperar_pose()
        x0, y0    = self.pose.x, self.pose.y
        msg       = Twist()
        msg.angular.z = 1.2   # giro constante durante toda la espiral
        velocidad = 0.2       # velocidad lineal inicial
        radio_max = 4.0       # detener al superar este radio desde el origen

        while True:
            rclpy.spin_once(self, timeout_sec=0.01)
            radio = math.sqrt((self.pose.x-x0)**2 + (self.pose.y-y0)**2)
            if radio > radio_max: break
            msg.linear.x = velocidad
            self.pub.publish(msg)
            velocidad += 0.003   # incremento suave en cada iteración
            time.sleep(0.05)
        self.pub.publish(Twist())

def main():
```

```
    rclpy.init()
    nodo = Espiral()
    nodo.dibujar()
    nodo.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__': main()
```

```
ros2 service call /reset std_srvs/srv/Empty
nano ~/espiral.py   # pegar el código, guardar con Ctrl+O y salir con Ctrl+X
python3 ~/espiral.py
```

> ★ **Conexión con el perfil trapezoidal**
> La espiral incrementa linealmente la velocidad en cada paso: es equivalente a la fase de aceleración constante de un perfil trapezoidal (Clase 9).

6. Dos Tortugas

Crear una segunda tortuga y controlar ambas demuestra la arquitectura multi-robot de ROS 2: cada robot tiene sus propios tópicos y un nodo controlador puede darles órdenes a todos.

6.1 Crear la segunda tortuga desde la terminal

```
ros2 service call /spawn turtlesim/srv/Spawn "{x: 3.0, y: 3.0, theta: 0.0, name:
'turtle2'}"
```

Aparece una segunda tortuga. Ahora hay dos tópicos de velocidad independientes: /turtle1/cmd_vel y /turtle2/cmd_vel. Ojo: turtle2 **no es un nodo nuevo**; vive dentro del mismo nodo /turtlesim, al que el servicio /spawn le agregó tópicos nuevos.

6.2 Control manual de dos tortugas

```
# Terminal A — turtle1 gira a la izquierda:
ros2 topic pub /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 2.0}, angular: {z:
1.0}}"

# Terminal B — turtle2 gira a la derecha:
ros2 topic pub /turtle2/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 2.0}, angular: {z: -
1.0}}"
```

6.3 Script Python — cuadrado y triángulo

Este script crea turtle2 por sí mismo; luego turtle1 dibuja un cuadrado y, al terminar, turtle2 dibuja un triángulo. Un único nodo controlador da órdenes a los dos robots.

```
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist
from turtlesim.msg import Pose
from turtlesim.srv import Spawn
import math, time
```

```
class DosTortugas(Node):
    def __init__(self):
        super().__init__('dos_tortugas')
        # Un publicador y una pose por tortuga, guardados por nombre
        self.pub  = {'turtle1': self.create_publisher(Twist, '/turtle1/cmd_vel', 10),
                     'turtle2': self.create_publisher(Twist, '/turtle2/cmd_vel', 10)}
        self.pose = {'turtle1': None, 'turtle2': None}
        self.create_subscription(Pose, '/turtle1/pose', self.cb1, 10)
        self.create_subscription(Pose, '/turtle2/pose', self.cb2, 10)
        self.spawn = self.create_client(Spawn, '/spawn')

    def cb1(self, msg): self.pose['turtle1'] = msg
    def cb2(self, msg): self.pose['turtle2'] = msg

    def crear_turtle2(self, x=8.0, y=8.0):
        self.spawn.wait_for_service()
        req = Spawn.Request()
        req.x = float(x); req.y = float(y)
        req.theta = math.pi; req.name = 'turtle2'
        futuro = self.spawn.call_async(req)
        rclpy.spin_until_future_complete(self, futuro)

    def esperar_poses(self):
        while self.pose['turtle1'] is None or self.pose['turtle2'] is None:
            rclpy.spin_once(self, timeout_sec=0.1)

    def avanzar(self, t, distancia, vel=1.5):
        x0, y0 = self.pose[t].x, self.pose[t].y
        msg = Twist()
        while True:
            rclpy.spin_once(self, timeout_sec=0.01)
            p = self.pose[t]              # SIEMPRE la pose mas reciente
            rest = distancia - math.sqrt((p.x-x0)**2 + (p.y-y0)**2)
            if rest <= 0: break
            msg.linear.x = min(vel, max(0.3, rest*2.0))
            self.pub[t].publish(msg)
        self.pub[t].publish(Twist()); time.sleep(0.1)

    def girar(self, t, angulo_rad):
        objetivo = self.pose[t].theta + angulo_rad
        msg = Twist()
        while True:
            rclpy.spin_once(self, timeout_sec=0.01)
            diff = objetivo - self.pose[t].theta   # pose mas reciente
            while diff >  math.pi: diff -= 2*math.pi
            while diff < -math.pi: diff += 2*math.pi
            if abs(diff) < 0.002: break
            vel = max(0.2, min(2.0, abs(diff)*2.5))
            msg.angular.z = vel if diff > 0 else -vel
            self.pub[t].publish(msg)
        self.pub[t].publish(Twist()); time.sleep(0.1)

    def dibujar(self):
        self.crear_turtle2(x=8.0, y=8.0)
        self.esperar_poses()
        # Turtle1: cuadrado
        for _ in range(4):
            self.avanzar('turtle1', 2.0)
            self.girar('turtle1', math.pi/2)
        # Turtle2: triangulo (empieza cuando turtle1 termina)
        for _ in range(3):
            self.avanzar('turtle2', 2.0)
            self.girar('turtle2', 2*math.pi/3)
```

```
def main():
    rclpy.init()
    nodo = DosTortugas()
    nodo.dibujar()
    nodo.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__': main()
```

```
ros2 service call /reset std_srvs/srv/Empty   # si turtle2 ya existe, el spawn falla
nano ~/dos_tortugas.py
python3 ~/dos_tortugas.py
```

> ⚠ **¿Por qué se lee self.pose[t] dentro del bucle?**
> Cada vez que llega un mensaje, el callback **reemplaza** la pose guardada por un objeto nuevo. Si una función guardara la pose una sola vez al empezar, seguiría mirando el objeto viejo, creería que la tortuga no se ha movido y el bucle nunca terminaría. Por eso, en cada vuelta se vuelve a leer la pose más reciente.

> ★ **Conexión con robótica real**
> En un sistema real con ROS 2, cada robot físico (brazo, vehículo, dron) publica y escucha sus propios tópicos. Un nodo planificador central les envía órdenes a todos: la misma arquitectura que acabas de ver con las dos tortugas.

7. Un Servicio Propio: ¿Dónde Está la Tortuga?

Para ver la diferencia entre tópico y servicio, este par de scripts ofrece la pose de la tortuga **solo cuando alguien la pide**. Usa el tipo estándar `std_srvs/srv/Trigger`: la petición va vacía y la respuesta trae `success` (verdadero o falso) y `message` (texto).

7.1 Servidor

Se suscribe al tópico /turtle1/pose (flujo continuo) y ofrece el servicio /donde_esta (una respuesta por pregunta).

```
# ============================================================
#  SERVIDOR de servicio: responde "donde esta la tortuga"
#  Clase 10 - Robotica Industrial FPTEN27 - UNIMET
# ============================================================
import math
import rclpy
from rclpy.node import Node
from turtlesim.msg import Pose
from std_srvs.srv import Trigger      # peticion vacia; respuesta: success (bool) +
message (texto)


class ServidorPose(Node):
    def __init__(self):
        super().__init__('servidor_pose')
```

```
        # 1) Escucha el TOPICO de la pose (flujo continuo, ~60 mensajes por segundo)
        self.pose = None
        self.create_subscription(Pose, '/turtle1/pose', self.cb_pose, 10)

        # 2) Ofrece un SERVICIO: solo responde cuando alguien pregunta
        self.create_service(Trigger, '/donde_esta', self.responder)
        self.get_logger().info('Servicio /donde_esta listo. Esperando preguntas...')

    def cb_pose(self, msg):
        self.pose = msg                  # siempre guarda la ultima pose recibida

    def responder(self, request, response):
        """Se ejecuta UNA vez por cada peticion que llega."""
        if self.pose is None:
            response.success = False
            response.message = 'Todavia no he recibido la pose de la tortuga'
        else:
            response.success = True
            response.message = (f'x = {self.pose.x:.2f}, y = {self.pose.y:.2f}, '
                                f'theta = {math.degrees(self.pose.theta):.1f} grados')
        self.get_logger().info(f'Me preguntaron. Respondi: {response.message}')
        return response


def main():
    rclpy.init()
    nodo = ServidorPose()
    rclpy.spin(nodo)          # queda vivo atendiendo topicos y peticiones hasta Ctrl+C
    nodo.destroy_node()
    rclpy.shutdown()


if __name__ == '__main__':
    main()
```

7.2 Cliente

Espera a que exista el servicio, envía la pregunta, espera la respuesta, la imprime y termina.

```
# ============================================================
#  CLIENTE de servicio: pregunta una vez "donde esta la tortuga"
#  Clase 10 - Robotica Industrial FPTEN27 - UNIMET
# ============================================================
import rclpy
from rclpy.node import Node
from std_srvs.srv import Trigger


def main():
    rclpy.init()
    nodo = Node('cliente_pose')
    cliente = nodo.create_client(Trigger, '/donde_esta')

    # Si el servidor no esta corriendo, no hay a quien preguntarle
    while not cliente.wait_for_service(timeout_sec=1.0):
        nodo.get_logger().info('Esperando a que aparezca el servicio /donde_esta...')

    # Enviar la peticion y ESPERAR la respuesta antes de seguir
    futuro = cliente.call_async(Trigger.Request())
    rclpy.spin_until_future_complete(nodo, futuro)

    respuesta = futuro.result()
    print('Exito:', respuesta.success)
    print('Respuesta:', respuesta.message)

    nodo.destroy_node()
    rclpy.shutdown()


if __name__ == '__main__':
    main()
```

7.3 Cómo probarlo

T1 — **El simulador**

```
ros2 run turtlesim turtlesim_node
```

T2 — **El servidor (queda esperando preguntas)**

```
python3 ~/servidor_pose.py
```

Cada vez que alguien pregunta, aparece una línea "Me preguntaron…".

T3 — **Mover la tortuga**

```
ros2 run turtlesim turtle_teleop_key
```

T4 **Preguntar**

```
python3 ~/cliente_pose.py
ros2 service call /donde_esta std_srvs/srv/Trigger
```

Con el cliente o directamente desde la terminal. Compara con ros2 topic echo /turtle1/pose, que no para de imprimir.

8. Cómo Leer el Grafo de Nodos

Con turtlesim, `dos_tortugas.py` corriendo y algunos comandos abiertos, `rqt_graph` puede mostrar nodos que no escribiste tú:

Nodo	Qué es
/turtlesim	El simulador. Las dos tortugas viven dentro de este único nodo.
/dos_tortugas	Tu script. Recibe las dos poses y envía las dos velocidades: el lazo cerrado dibujado con flechas.
/_ros2cli_ + número	Un comando de terminal que sigue corriendo, como `ros2 topic echo` o `ros2 topic pub`. Cada uno crea un nodo temporal [9]. Desaparece al cerrarlo con Ctrl+C.
/_ros2cli_daemon_…	El daemon de ROS 2: un proceso de fondo que guarda la información del grafo para que comandos como `ros2 node list` respondan rápido. Existe porque en ROS 2 no hay un nodo central que lleve la lista [9].
/rqt_gui_py_node_ + número	La propia ventana de rqt_graph, que se une a la red para poder ver a los demás.

9. Instalación del Entorno en tu Computador

Instrucciones para instalar WSL2 + Ubuntu 22.04 + ROS 2 Humble en Windows.

⚠ Requisitos

Windows 10 versión 2004 o superior / Windows 11. Conexión a internet (~4 GB de descarga). Al menos 15 GB libres en disco.

1 **Abrir PowerShell como Administrador**

Tecla Windows → escribe "PowerShell" → clic derecho → "Ejecutar como administrador".

2 **Instalar WSL2 con Ubuntu 22.04**

```
wsl --install -d Ubuntu-22.04
```

Al finalizar crea usuario y contraseña. Anótalos.

3 **Verificar la versión de Ubuntu**

```
lsb_release -a
```

Debe decir "Codename: jammy". Cualquier otro resultado: consulta al profesor.

9.1 Instalar ROS 2 Humble

Ejecuta estos comandos en orden dentro de Ubuntu 22.04:

```
# 1. Actualizar el sistema
sudo apt update && sudo apt upgrade -y

# 2. Instalar dependencias
sudo apt install software-properties-common curl -y

# 3. Agregar clave de ROS 2
sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key \
  -o /usr/share/keyrings/ros-archive-keyring.gpg

# 4. Agregar repositorio
echo "deb [arch=$(dpkg --print-architecture) \
  signed-by=/usr/share/keyrings/ros-archive-keyring.gpg] \
  http://packages.ros.org/ros2/ubuntu \
  $(. /etc/os-release && echo $UBUNTU_CODENAME) main" \
  | sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null

# 5. Instalar ROS 2 Humble (~5-10 minutos)
sudo apt update
sudo apt install ros-humble-desktop -y

# 6. Configurar entorno automático
echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc
source ~/.bashrc

# 7. Instalar turtlesim
sudo apt install ros-humble-turtlesim -y

# 8. Verificar (debe decir 'All 5 checks passed')
ros2 doctor
```

9.2 Manejar tus scripts en WSL

Los scripts se guardan en tu carpeta personal de Ubuntu (`~`). Desde Ubuntu, las unidades de Windows están montadas en `/mnt/`; por ejemplo, la unidad C: es `/mnt/c` [11].

Tarea	Comando
Ver qué scripts tienes	`ls -l ~/*.py`
Ver el contenido sin modificarlo	`cat ~/cuadrado.py`
Editar (guardar Ctrl+O, salir Ctrl+X)	`nano ~/cuadrado.py`
Ver tu usuario de Windows	`ls /mnt/c/Users/`
Copiar un archivo descargado en Windows	`cp /mnt/c/Users/TuUsuario/Downloads/archivo.py ~/`
Abrir tu carpeta de Ubuntu en el Explorador de Windows	`explorer.exe ~` [11]
Guardar una copia de respaldo	`mv ~/archivo.py ~/archivo_viejo.py`
Borrar un script (pide confirmación)	`rm -i ~/archivo.py`

> ⚠ **rm no usa papelera**
> El archivo se borra definitivamente. Verifica el nombre con `ls` antes de borrar, o usa `rm -i` para que pida confirmación. El manual oficial del comando se consulta con `man rm` (salir con q).

9.3 Solución de problemas

Problema	Solución
"ros2: command not found"	`source /opt/ros/humble/setup.bash`
"Unable to locate package ros-humble-…"	Verifica que sea Ubuntu 22.04 (jammy). Otras versiones no son compatibles.
La ventana de turtlesim no aparece	Requiere Windows 11 o Windows 10 actualizado. Ejecuta `wsl --update` en PowerShell.
Un script no dibuja nada	Verifica que `turtlesim_node` esté corriendo en otra terminal antes del script.
El spawn de turtle2 falla	Ya existe una tortuga con ese nombre: ejecuta `ros2 service call /reset std_srvs/srv/Empty`.
`ros2 node list` no muestra un nodo que sí corre	Reinicia el daemon: `ros2 daemon stop` y luego `ros2 daemon start` [9].
"No such file" al copiar desde Windows	Revisa el nombre exacto con `ls /mnt/c/Users/TuUsuario/Downloads/`; si tiene espacios o paréntesis, ponlo entre comillas.
Ubuntu 22.04 no abre	Búscalo como "Ubuntu 22.04" en el menú de inicio, no como "Ubuntu" genérico.

10. Referencias

[1] Lynch, K. M. & Park, F. C. (2017). *Modern Robotics: Mechanics, Planning, and Control*. Cambridge University Press. Apéndice B (B.1 y B.3) y secciones 5.3–5.4.

[2] Correll, N. *Introduction to Autonomous Robots* (LibreTexts). Sección 3.1.4, "Other representations for orientation".

[3] Herath, D. & St-Onge, D. (eds.) (2022). *Foundations of Robotics: A Multidisciplinary Approach with Python and ROS*. Springer. Sección 6.4.3, "Quaternion".

[4] ABB. *Technical reference manual — RAPID Instructions, Functions and Data types* (3HAC16581-1); y foros oficiales ABB Robotics Community y RobotStudio (tech-community.robotics.abb.com, forums.robotstudio.com).

[5] FZI Forschungszentrum Informatik. *Robot interface proposal* — páginas KUKA y ABB (fzi-robot-interface-proposal.readthedocs.io).

[6] University of Texas at Austin. Wiki de programación de robots KUKA (cloud.wikis.utexas.edu).

[7] Basler. *KUKA XYZ-ABC format* (docs.baslerweb.com/stereovisard/rc_cube/en/pose_format_kuka).

[8] RoboDK. Documentación de la API de Python, ejemplos oficiales y foro oficial (robodk.com/doc/en/PythonAPI, robodk.com/forum).

[9] Open Robotics. *ROS 2 Documentation*: Understanding nodes, topics y services; About command line tools; Synchronous vs. asynchronous service clients (docs.ros.org).

[10] ROS Answers. "turtlesim velocity units", respuesta sobre el tiempo límite de 1 s de las órdenes de velocidad (answers.ros.org/question/12516).

[11] Microsoft Learn. *Working across Windows and Linux file systems* (learn.microsoft.com/windows/wsl/filesystems).